

PACE: A Piece-Wise Approximate Floating-Point Divider with Runtime Configurability and High Energy Efficiency

CHENYI WEN, Zhejiang University, Hangzhou, China

HAONAN DU, Zhejiang University, Hangzhou, China

JIAYI WANG, Zhejiang University, Hangzhou, China

ZHENG Rui CHEN, Zhejiang University, Hangzhou, China

LI ZHANG, Hubei University of Technology, Wuhan, China

QI SUN, Zhejiang University, Hangzhou, China

CHENG ZHUO, Zhejiang University, Hangzhou, China

Approximate computing emerges as a viable solution to enhance energy efficiency in applications sensitive to human perception, particularly on edge devices. This work introduces a novel piece-wise approximate floating-point divider that boasts resource efficiency and runtime configurability. Our method leverages a piece-wise approximation algorithm for computing $1/y$ by exploiting powers of 2, complemented by an error compensation technique grounded in thorough mathematical analysis. This approach facilitates the realization of a reciprocal-based floating-point divider devoid of multipliers, which not only mitigates hardware resource consumption but also reduces latency. Additionally, we unveil a multi-level runtime configurable hardware architecture that significantly improves flexibility across diverse application contexts. Compared to the existing state-of-the-art approximate dividers and truncated exact dividers, our proposed solution achieves a superior compromise between precision and resource efficiency. Application-level evaluations reveal that our design provides over 87.7% energy saving while maintaining a negligible impact on output quality.

CCS Concepts: • **Hardware** → *Combinational circuits; Arithmetic and datapath circuits;*

Additional Key Words and Phrases: Approximate computing, piece-wise, divider, floating-point, energy efficiency

ACM Reference Format:

Chenyi Wen, Haonan Du, Jiayi Wang, Zhengrui Chen, Li Zhang, Qi Sun, and Cheng Zhuo. 2025. PACE: A Piece-Wise Approximate Floating-Point Divider with Runtime Configurability and High Energy Efficiency. *ACM Trans. Des. Autom. Electron. Syst.* 30, 2, Article 21 (January 2025), 23 pages. <https://doi.org/10.1145/3706634>

Chenyi Wen and Haonan Du contributed equally to this research.

The authors thank the supports from NSFC (Grant No. 62034007, 62304074 and W2412034), and Zhejiang University Education Foundation Qizhen Scholar Foundation.

Authors' Contact Information: Chenyi Wen, Zhejiang University, Hangzhou, China; e-mail: wwency@zju.edu.cn; Haonan Du, Zhejiang University, Hangzhou, China; e-mail: hndu@zju.edu.cn; Jiayi Wang, Zhejiang University, Hangzhou, China; email: wwangjy@zju.edu.cn; Zhengrui Chen, Zhejiang University, Hangzhou, China; e-mail: zr.chen@zju.edu.cn; Li Zhang, Hubei University of Technology, Wuhan, China; e-mail: zhangli@hbut.edu.cn; Qi Sun, Zhejiang University, Hangzhou, China; e-mail: qisunchn@zju.edu.cn; Cheng Zhuo (Corresponding author), Zhejiang University, Hangzhou, China; e-mail: czhuo@zju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1084-4309/2025/01-ART21

<https://doi.org/10.1145/3706634>

1 Introduction

Recently, with a notable surge of human perception-related tasks (e.g., audio/image processing, machine learning), it has become evident that stringent precision and exactness are not always necessary [5, 18, 31, 34]. Simultaneously, the rapid expansion of the **Internet of Things (IoT)** and edge computing technologies has spotlighted the critical need for optimizing energy consumption, given the inherent hardware limitations. This context has fueled interest in approximate computing, an approach that prioritizes energy efficiency by compromising computational accuracy [9, 22].

In the quest for reduced energy consumption, the exploration of approximate arithmetic units, including adders and multipliers, has shown potential in lowering power usage while retaining acceptable levels of accuracy [15, 32, 33]. Division, however, stands out as an operation that typically requires more time, energy, and hardware resources than its counterparts [29]. For example, an exact **floating-point (FP)** divider consumes over $15\times$ more **Area-Delay-Product (ADP)** and exhibits more than $1.5\times$ higher power consumption compared to an exact FP multiplier [28]. Nevertheless, division plays a crucial role in a wide array of applications, from the OpenCV multimedia library [4] to page-ranking normalization [12] and robotics control algorithms [8]. Therefore, **the demand for resource-efficient dividers is significant**, especially in contexts where division is indispensable.

Several studies have proposed various approximate divider designs. We can group them into three main categories: subtractor-array-based [6, 7, 11, 14], logarithm-based [13, 19, 21, 27, 28], and reciprocal-based dividers [3, 17, 20, 25, 29, 35]. While each type has its advantages, they also come with limitations. Subtractor-array and reciprocal-based dividers can be very accurate, but they often require substantial hardware overhead. Subtractor-array dividers encounter latency issues due to their array structure [6, 7, 11, 14]. Reciprocal-based dividers always need one to three multipliers to complete computations [3, 17, 20, 25, 35]. Despite these challenges, the above dividers still manifest benefits in terms of hardware resource efficiency, especially when considering the inherent complexity differences between multipliers and dividers. On the other hand, logarithm-based dividers are more hardware-efficient but encounter difficulties in maintaining accuracy and offering runtime configurability. They often rely on pre-loaded **lookup tables (LUTs)** for error compensation, which significantly increases hardware resource requirements and makes runtime adjustments impractical [13, 19].

Obviously, designing an efficient and flexible approximate divider encompasses several challenges, as outlined below:

- A prevalent issue with many current approximate division techniques is their **lack of circuit-friendly design implementations**. As a consequence, these approaches are often constrained by the inherent latency of subtractor arrays or the excessive hardware overheads associated with multiple multipliers.
- **Improving computational precision or incorporating error correction mechanisms proves challenging** across existing approximate division methods. Attempts to reduce computational inaccuracies generally result in additional computational cycles or the need for substantial hardware enhancements, such as the incorporation of LUTs. These measures frequently negate the intended energy efficiency gains of approximate computing.
- **The need for runtime configurability is frequently disregarded**. Many existing designs either do not support runtime adjustments or introduce prohibitive time and energy costs for such modifications. This limitation significantly constrains the versatility and practical utility of these approximate dividers.

To tackle the aforementioned challenges, we propose PACE (**P**iece-wise **A**pproximate and **C**onfigurab**L**E FP divider), a resource-efficient and runtime configurable design. Our major contributions include:

- We have developed a **multiplier-free reciprocal-based floating-point divider** that substitutes the multiplication by $1/y$ with simpler operations such as shifts and additions, thereby reducing hardware resource consumption and decreasing latency.
- We present a **piece-wise approximation algorithm** for the $1/y$ operation. This approach enhances the precision of division by increasing the number of piece-wise linear segments to approximately $1/y$, thereby improving division accuracy with minimal complexity.
- We propose an **error compensation technique** for our approximation algorithm, formulated through detailed mathematical analysis. This technique can be seamlessly incorporated into the algorithm without necessitating extra hardware resources.
- Lastly, we have crafted a **multi-level runtime configurable circuit** to boost flexibility for various applications. This innovative structure allows for dynamic adjustments of the approximation level during operation, enabling tradeoff between delay and energy consumption.

The experimental results demonstrate that PACE achieves a favorable tradeoff between accuracy and resource utilization when compared to the other leading dividers in the field. Notably, PACE matches the accuracy of the most precise approximate divider, QIAD [17], while only requiring 10.75% of the resources according to the ADP metric. Among alternatives, FaNZed [28] is the only divider that demands fewer hardware resources than PACE, yet it incurs errors approximately 5× greater than those of PACE. At the application level, PACE exhibits only minimal degradation in output quality, with average PSNR reductions of just 0.09 dB, 0.63 dB, and 0.27 dB for K-means color quantization, JPEG compression, and Gaussian filtering, respectively. Moreover, it consumes merely 12.3%, 9.5%, and 9.8% of the energy compared to an exact divider.

This article is an extension to our prior work [30], including an enhanced error compensation technique, an additional truncation mechanism, and more comprehensive experimental results. The rest of the article is organized as follows: Section 2 provides a concise review of exact floating-point division and the current state of research on approximate dividers. Sections 3 and 4 elaborate on the proposed approximate division algorithm and its error compensation method, respectively. The design of the multiplier-free divider is discussed in Section 5. Section 6 evaluates the accuracy, resource efficiency, and performance of our design in comparison with existing work. Section 7 evaluates the impact of integrating PACE into real-world applications. Potential limitations of PACE are discussed in Section 8, and concluding remarks are presented in Section 9.

2 Background

2.1 FP Division

According to the IEEE 754 standard [1] for FP arithmetic, any FP number F_x can be represented as Equation (1):

$$F_x = (-1)^{S_x} \cdot x \cdot 2^{E_x}, \quad (1)$$

where S_x , x , and E_x refer to the sign, mantissa, and exponent of the FP number F_x , with $S \in \{0, 1\}$ and $x \in [1, 2)$ with the hidden one included.

The division of two FP numbers F_x and F_y can be calculated as Equation (2), where $\oplus$ is the XOR operation:

$$\frac{F_x}{F_y} = \begin{cases} (-1)^{(S_x \oplus S_y)} \cdot \frac{x}{y} \cdot 2^{E_x - E_y} & , 1 \leq \frac{x}{y} < 2 \\ (-1)^{(S_x \oplus S_y)} \cdot 2 \frac{x}{y} \cdot 2^{E_x - E_y - 1} & , \frac{1}{2} < \frac{x}{y} < 1 \end{cases} \quad (2)$$

In Equation (2), computing the sign and exponent is relatively straightforward. The methodology for calculating the mantissa is similar to that of integer division. Consider the division $N/D = Q$ with remainder R , where N , D , Q , and R denote the dividend, divisor, quotient, and remainder, respectively. The classic restoring division algorithm, as represented by Algorithm 1, elaborates on this process. At each iteration, the algorithm attempts to subtract the divisor from the current remainder to ascertain the appropriate bit for the quotient. If the remainder becomes negative, indicating the subtraction is not possible, the divisor is added back to “restore” the original value.

ALGORITHM 1: Restoring Division

Require: N, D, k

Ensure: Q, R

```

1:  $R^{(k)} \leftarrow N$ 
2:  $D \leftarrow D \ll k$  {left shift}
3: for  $i \leftarrow k - 1$  down to 0 do
4:    $D \leftarrow D \gg 1$ 
5:    $R^{(i)} \leftarrow R^{(i+1)} - D$ 
6:   if  $R^{(i)} \geq 0$  then
7:      $q_i \leftarrow 1$ 
8:   else
9:      $q_i \leftarrow 0$ 
10:     $R^{(i)} \leftarrow R^{(i)} + D$  {Restoring}
11:   end if
12: end for

```

The non-restoring division method differs by always retaining the result of the subtraction, even when it results in an incorrect partial remainder. Instead of restoring the previous state, this approach compensates by altering the next operation to addition, if the previous subtraction yields a negative remainder. Quotient bits are determined from the set $\{1, -1\}$, where 1 indicates subtraction and -1 for addition, necessitating a subsequent conversion to normalize both the remainder and the quotient. An advancement in this method is the SRT division [10], which reincorporates 0 (indicating no operation) into the quotient bit selection set. This inclusion permits the omission of operations on certain iterations by directly shifting the partial remainder, thereby streamlining the division process for improved efficiency.

Algorithm 1 and its variants can be easily transformed into iterative divider circuits, which function across multiple clock cycles. Alternatively, unfolding these iterations into a cell array facilitates the creation of a purely combinational divider. Figure 1 illustrates the core structure of a non-restoring array divider. This array consists of $k \times k$ cells, each capable of executing addition or complementary subtraction. However, such dividers incur resource consumption proportional to $O(k^2)$ for k -bit quotients [26].

The above discussions represent the digit recurrence division approach, favored for standalone hardware implementations [23]. Other division algorithms fall into distinct categories, such as functional iteration, high radix, table lookup, and their hybrids [24]. While these methods introduce certain enhancements, they generally do not deviate from the fundamental complexity associated with division. Throughout the remainder of this article, we consider the implementation of an exact divider in the form of an array divider, given that the dividers examined later are primarily combinational.

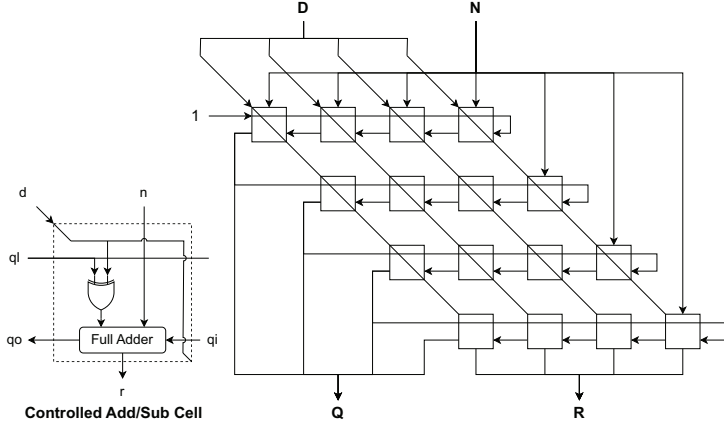


Fig. 1. Core structure of a non-restoring array divider.

2.2 Approximate Divider

Given the substantial energy expenditure and computational complexity associated with exact dividers, numerous researchers have shifted their focus to approximate dividers, exchanging efficiency for accuracy. Here we summarize several common approximation strategies:

Direct modification to the exact divider. This is a relatively intuitive strategy. Researches cited in [11, 14] proposed a pruning mechanism on conventional dividers, diminishing the bit width of the input following the detection of the leading one, thus simplifying the division procedure. Alternatively, studies such as [2, 6, 7] suggest the partial substitution of standard units with approximate ones in restoring, non-restoring, and high-radix dividers. This modification permits adjustable levels of approximation. However, these modifications are still constrained by the underlying principles of precise division, inheriting its hardware redundancy while possessing less-than-ideal approximation performance.

Utilize the logarithmic conversion. A typical logarithm-based approximate divider undergoes three phases: logarithmic transformation, subtraction, and inverse logarithmic conversion. In this way, the division is transformed into a simpler subtraction in the logarithmic domain. To facilitate binary computations, logarithmic operations are usually approximated as linear functions, known as the Mitchell Method [21], outlined in Equation (3):

$$\log N = k + \log(1 + x) \approx k + x, \quad (3)$$

where $N = 2^k(1 + x)$, $0 \leq x < 1$. Despite its simplicity, the original Mitchell Method can introduce non-negligible errors. Several studies introduced enhancements based on it, such as FaNZeD [28] and LEAD [27], which employ compensation coefficients and exponential series expansions, respectively, to refine accuracy.

Convert to multiplication with the reciprocal. Given that multiplication typically requires less resources [17], another approach to calculating the division involves converting the operation A/B into $A \times (1/B)$. Several techniques for calculating the reciprocal $1/B$ have been explored. For instance, ILAFD [25] applies direct bit-wise inversion, while TruncApp [29] adopts a specific approximation method shown in Equation (4):

$$\frac{1}{X_B} \approx \left\lceil \frac{X_B + 1}{2} \right\rceil, \quad (4)$$

where $X_B \in [1, 2)$ represents the normalized fractional part of B , and the overline indicates bit-wise inversion. SEERAD [35] computes optimal parameters for transforming the reciprocal into multiplication and shift operations in advance.

While straightforward, these methods often yield higher errors and generally rely on LUTs for parameter storage or error compensation. Therefore, recent reciprocal-based studies have delved into mathematical strategies for approximating $1/x$. For example, Taylor expansions, Newton–Raphson iterations, and quadratic curve fitting are employed in SAADI-EC [20], ADN-EC [3], and QIAD [17], respectively. These approaches achieve notable precision but usually require either several multipliers within a single cycle or additional cycles if limited to one multiplier. Such requirements increase the complexity of the divider design.

In conclusion, although significant progress has been made in the field, finding an approximate divider that optimally balances precision with resource efficiency continues to be a formidable challenge.

3 Approximate Division Design

3.1 Approximate Division Algorithm

Given that the computation of the mantissa x/y is notably more energy and time-intensive than other components in the FP division, our focus shifts towards designing it for energy efficiency. To address the computational complexity of x/y , we introduce a transformation as shown in Equation (5):

$$x/y = (1/y) \times (x - y) + 1. \quad (5)$$

Here, we approximate $1/y$ based on the value of y , utilizing a linear plane to interpolate the x/y surface as defined in Equation (5). Equation (5) is preferred for division because multiplying $1/y$ by a smaller value minimizes potential numerical errors. Given $x \in [1, 2)$ and $(x - y) \in (-1, 1)$, $|x| > |x - y|$, making $(x - y)$ a more suitable choice than x for multiplication. Assume y lies within the range $[y_1, y_2)$, where y_1 and y_2 are given by

$$y_1 = 1 + f_1 = 1 + \frac{t}{2^n}, \quad (6)$$

$$y_2 = 1 + f_2 = 1 + \frac{t+1}{2^n}. \quad (7)$$

Here, f_1 and f_2 fall within $[0, 1)$. And n represents the approximation level. Variable t is a binary string constructed from the first n bits of the fractional part of y . For instance, with $n = 3$ and $y = 1.01011 \dots$, t becomes 010 in binary (which is 2 in decimal).

The average value of $1/y$ within $[y_1, y_2)$ is denoted as y_n^t :

$$y_n^t = \frac{1}{y_2 - y_1} \int_{y_1}^{y_2} \frac{1}{y} dy = \frac{\ln y_2 - \ln y_1}{y_2 - y_1}. \quad (8)$$

To further simplify Equation (8), we utilize the Taylor series expansion of $\ln(1 + f)$, as shown in Equation (9):

$$\ln(1 + f) = 1 + f - \frac{f^2}{2} + \frac{f^3}{3} - \dots + (-1)^{k+1} \frac{f^k}{k}. \quad (9)$$

Substituting Equation (9) into Equation (8), we get

$$\begin{aligned} y_n^t &= \frac{\ln(1 + f_2) - \ln(1 + f_1)}{(1 + f_2) - (1 + f_1)} = \frac{1}{f_2 - f_1} \times \left(f_2 - f_1 - \frac{f_2^2 - f_1^2}{2} + \frac{f_2^3 - f_1^3}{3} - \dots \right) \\ &= 1 - \frac{f_2 + f_1}{2} + \frac{f_2^2 + f_1^2 + f_1 f_2}{3} - \dots \end{aligned} \quad (10)$$

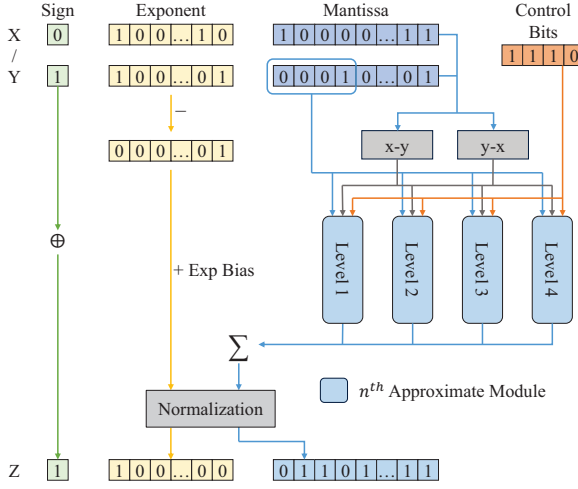


Fig. 2. Architecture of the proposed approximate divider.

Focusing on the first two terms of Equation (10), i.e., 1 and $-\frac{f_2+f_1}{2}$, and approximating subsequent terms as $\frac{f_1 f_2}{2}$, this simplification aligns with hardware implementation convenience, capturing the primary behavior of the series. The approximated expression for y_n^t , denoted as k_n^t , is

$$y_n^t \approx k_n^t = 1 - \frac{f_2 + f_1}{2} + \frac{f_1 f_2}{2} = 1 - \frac{2t + 1}{2^{n+1}} + \frac{t^2 + t}{2^{2n+1}}, \quad (11)$$

where n and t are as previously defined. By integrating this approximation into Equation (5), we derive Equation (12):

$$x/y \approx y_n^t \times (x - y) + 1 \approx k_n^t \times (x - y) + 1. \quad (12)$$

For additional analysis, let f_{approx}^n symbolize the approximate mantissa division at the n th level of approximation:

$$f_{approx}^n = k_n^t \times (x - y) + 1. \quad (13)$$

Through these mathematical expressions, we lay the groundwork for both computation- and energy-efficient approach to approximate mantissa division.

3.2 Multi-Level Approximate Division Computation

A multi-level computation design is proposed to implement the approximation algorithm, as depicted in Figure 2. It starts by computing both the difference between the inputs ($x - y$ and $y - x$) using subtractors. Then, it feeds these differences along with the original mantissa y and control bits into a multi-level unit. The first level generates an initial approximation f_{approx}^1 and subsequent levels refine this estimate by correcting deviations. This multi-layer structure allows for flexible adjustments at runtime by simply choosing the desired number of active levels.

To facilitate further discussion, we define f_{ec}^n as the error correction applied between two consecutive approximate levels n and $n - 1$. In other words, f_{ec}^n represents the correction term calculated at the n th level. As illustrated in Figure 2, this architecture allows for easy configuration of accuracy. We achieve this by controlling the granularity of approximation, which is explained as follows:

$$f_{approx}^n = f_{approx}^1 + \sum_{i=2}^n f_{ec}^i. \quad (14)$$

Table 1. Relationship between the First 4 Digits of y Mantissa and K_n^t

$y[1:4]^*$	K_2^t	K_3^t	K_4^t
0000	$K_2^0 = 2^{-3}$	$K_3^0 = 2^{-4}$	$K_4^0 = 2^{-5}$
0001			$K_4^1 = -2^{-5} + 2^{-8}$
0010		$K_3^1 = -2^{-4} + 2^{-6}$	$K_4^2 = 2^{-5} - 2^{-8}$
0011			$K_4^3 = -2^{-5} + 2^{-7}$
0100	$K_2^1 = -2^{-4}$	$K_3^2 = 2^{-4} - 2^{-6}$	$K_4^4 = 2^{-5} - 2^{-7}$
0101			$K_4^5 = -2^{-6} - 2^{-8}$
0110		$K_3^3 = -2^{-5}$	$K_4^6 = 2^{-6} + 2^{-8}$
0111			$K_4^7 = -2^{-6}$
1000	$K_2^2 = 2^{-4}$	$K_3^4 = 2^{-5}$	$K_4^8 = 2^{-6}$
1001			$K_4^9 = -2^{-7} - 2^{-8}$
1010		$K_3^5 = -2^{-6}$	$K_4^{10} = 2^{-7} + 2^{-8}$
1011			$K_4^{11} = -2^{-7}$
1100	$K_2^3 = 0$	$K_3^6 = 2^{-6}$	$K_4^{12} = 2^{-7}$
1101			$K_4^{13} = -2^{-8}$
1110		$K_3^7 = 0$	$K_4^{14} = 2^{-8}$
1111			$K_4^{15} = 0$

*The first 4 digits of y mantissa.

The base approximation f_{approx}^1 can be calculated using Equations (11) and (13):

$$f_{approx}^1 = \begin{cases} 1 + \frac{1}{2}(x - y) + \frac{1}{4}(x - y), & y < 1.5 \\ 1 + \frac{1}{2}(x - y), & y \geq 1.5 \end{cases}. \quad (15)$$

Then we can calculate f_{ec}^n based on Equation (13):

$$f_{ec}^n = \left(k_n^t - k_{n-1}^{floor(t/2)} \right) \times (x - y), \quad (16)$$

where $floor(\cdot)$ denotes the round-down function. To simplify our discussion, we define K_n^t as the correction applied to k_n^t between neighboring levels:

$$K_n^t = k_n^t - k_{n-1}^{floor(t/2)} = \begin{cases} \frac{1}{2^{n+1}} - \frac{t}{2^{2n+1}}, & t \bmod 2 = 0 \\ -\frac{1}{2^{n+1}} + \frac{t+1}{2^{2n+1}}, & t \bmod 2 = 1 \end{cases}. \quad (17)$$

The architecture leverages a key advantage: all K_n^t values are exclusively powers of 2. This simplifies calculating f_{ec}^n significantly, as it only requires basic shift and addition operations. For instance, determining K_n^t for $n = 2, 3, 4$ depends solely on the first 4 bits of y 's mantissa, as detailed in Table 1.

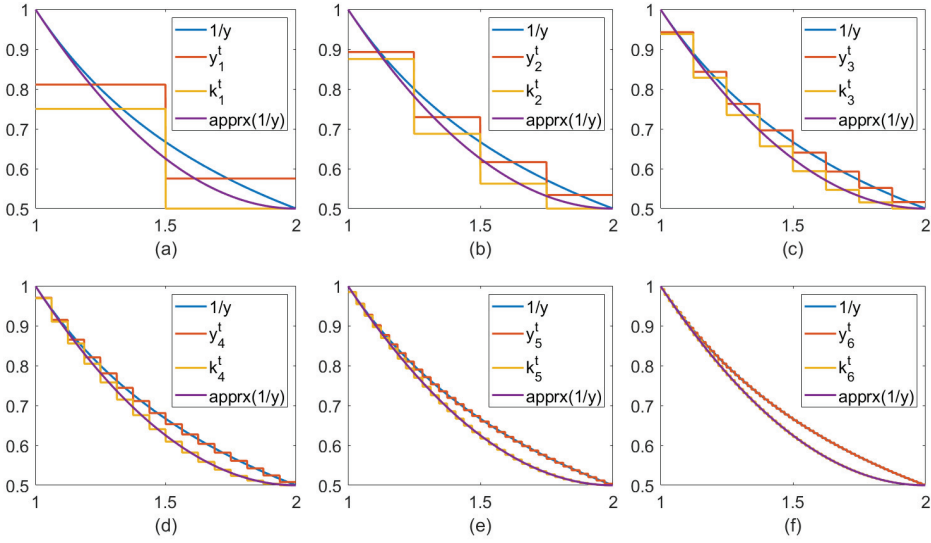


Fig. 3. Values of $1/y$, y_n^t , k_n^t , and $\text{appr}(1/y)$ for different approximate levels n , where y is in the range of $[1,2]$: (a) approximate level $n = 1$; (b) approximate level $n = 2$; (c) approximate level $n = 3$; (d) approximate level $n = 4$; (e) approximate level $n = 5$; (f) approximate level $n = 6$.

4 Error Analysis and Compensation

4.1 Limitation of the Proposed Algorithm

While the accuracy of the proposed approximation algorithm improves with increasing approximation levels n , achieving exact computational accuracy remains elusive due to the algorithm's intrinsic limitations. Specifically, as n approaches infinity, the function k_n^t , representing $\text{appr}(1/y)$, diverges from the true $1/y$ function:

$$\begin{aligned} \text{appr}(1/y) &= \lim_{n \rightarrow \infty} k_n^t = \lim_{n \rightarrow \infty} \left(1 - \frac{2t+1}{2^{n+1}} + \frac{t^2+t}{2^{2n+1}} \right) \\ &= 0.5y^2 - 2y + 2.5 \end{aligned} \quad (18)$$

Figure 3 illustrates the comparison of values for $1/y$, y_n^t , k_n^t , and $\text{appr}(1/y)$ as the approximation level n varies from 1 to 6. It is evident that the alignment between k_n^t and y_n^t progressively improves from levels 1 to 4. However, notably, this alignment does not exhibit further enhancement at levels 5 and 6. This indicates the necessity for error compensation mechanisms at the fourth or lower level of approximation.

4.2 Error Analysis

Here, we use **Mean-Square-Error (MSE)** as the error metric to obtain the following to quantify the approximation error:

$$\begin{aligned} \text{MSE} &= \int_1^2 \int_1^2 (x/y - f_{\text{approx}}^n)^2 dx dy = \int_1^2 \int_1^2 \left(\frac{1}{y} - k_n^t \right)^2 (x - y)^2 dx dy \\ &= \frac{1}{3} \sum_{t=0}^{2^n-1} \int_{1+\frac{t}{2^n}}^{1+\frac{t+1}{2^n}} \left(\frac{1}{y} - k_n^t \right)^2 (7 - 9y + 3y^2) dy = \frac{1}{3} \sum_{t=0}^{2^n-1} [A(k_n^t)^2 + Bk_n^t + C] \end{aligned} \quad (19)$$

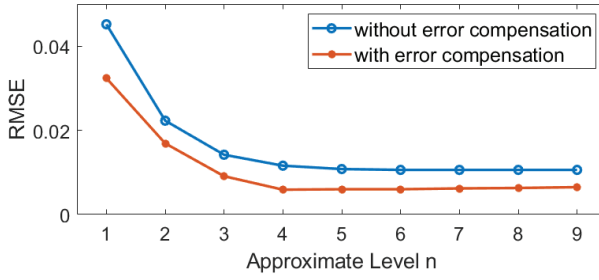


Fig. 4. Values of *RMSE* (with/without error compensation) when approximate level n varies.

where A , B , and C are coefficients determined by n and t :

$$A = \frac{1}{2^{3n}} [3t^2 + 3(1 - 2^n)t + 2^{2n} + 1 - 2^n - 2^{n-1}], \quad (20)$$

$$B = -14 \ln \frac{2^n + t + 1}{2^n + t} + \frac{12}{2^n} - \frac{6t + 3}{2^{2n}}, \quad (21)$$

$$C = \frac{3}{2^n} + \frac{7/2^n}{(1 + t/2^n)(1 + (1 + t)/2^n)} - 9 \ln \frac{2^n + t + 1}{2^n + t}. \quad (22)$$

Based on Equation (19), the **Root Mean Square Error (RMSE)** can be calculated as $MSE^{0.5}$ at each approximation level. Figure 4 shows that *RMSE* significantly decreases as n increases from 1 to 4. However, beyond $n = 4$, the error plateaus, indicating that further increases in approximation level do not contribute to improved accuracy. Consequently, we hope to further develop error compensation methods to improve calculation accuracy.

4.3 Error Compensation

Given A is quadratic in t , its minimum occurs at $t = 2^{n-1}$ or $t = 2^{n-1} - 1$:

$$A \geq \frac{1}{2^{3n}} (5 \times 2^{2n-2} + 1) > 0. \quad (23)$$

Since $A > 0$, minimizing *MSE* requires k_n^t at its optimal value. We can introduce c_n^t to adjust k_n^t , which gives KC_n^t :

$$KC_n^t = k_n^t + c_n^t = -\frac{B}{2A}, \quad (24)$$

$$MSE_{min} = \int_1^2 \int_1^2 \left(\frac{1}{y} - KC_n^t \right)^2 (x - y)^2 dx dy. \quad (25)$$

RMSE results with error compensation are presented in Figure 4. Specifically, we observe that compensating for L1 and L2 does not yield the same accuracy gains as moving to L2 or L3. However, compensating at L3 and L4 is more effective than advancing to L4 and L5. For levels above 4, neither error compensation nor level improvement significantly improves accuracy. Therefore, our hardware implementation focuses on the first four levels, with error compensation applied only to levels 3 and 4. Table 2 lists $c_n^t = -\frac{B}{2A} - k_n^t$ for $n = 3, 4$, in binary notation.

To achieve both high accuracy and minimal hardware overhead, we set the following guidelines for the values of c_n^t :

- To reduce hardware complexity, c_n^t should be a power of 2, allowing $c_n^t(x - y)$ computation via a simple shift operation.

Table 2. Relationship between the First 4 Digits of y Mantissa and c_n^t

$y[1:4]^*$	c_3^t	c_4^t
0000	$c_3^0 = 0.000000\mathbf{10}...$	$c_4^0 = 0.000000\mathbf{01}...$
0001		$c_4^1 = 0.000000\mathbf{01}...$
0010	$c_3^1 = 0.00000\mathbf{100}...$	$c_4^2 = 0.00000\mathbf{010}...$
0011		$c_4^3 = 0.00000\mathbf{100}...$
0100	$c_3^2 = 0.00000\mathbf{111}...$	$c_4^4 = 0.00000\mathbf{110}...$
0101		$c_4^5 = 0.00000\mathbf{111}...$
0110	$c_3^3 = 0.0000\mathbf{1010}...$	$c_4^6 = 0.0000\mathbf{1001}...$
0111		$c_4^7 = 0.0000\mathbf{1010}...$
1000	$c_3^4 = 0.0000\mathbf{1011}...$	$c_4^8 = 0.0000\mathbf{1011}...$
1001		$c_4^9 = 0.0000\mathbf{1011}...$
1010	$c_3^5 = 0.0000\mathbf{1011}...$	$c_4^{10} = 0.0000\mathbf{1011}...$
1011		$c_4^{11} = 0.0000\mathbf{1010}...$
1100	$c_3^6 = 0.0000\mathbf{1000}...$	$c_4^{12} = 0.0000\mathbf{1001}...$
1101		$c_4^{13} = 0.0000\mathbf{0111}...$
1110	$c_3^7 = 0.000000\mathbf{11}...$	$c_4^{14} = 0.00000\mathbf{101}...$
1111		$c_4^{15} = 0.000000\mathbf{01}...$

*The first 4 digits of y mantissa.

- To ensure accuracy, the value of c_n^t should be retained in or near the most-significant-bit, similar to the position highlighted in bold in Table 2.

Therefore, we can choose the c_n as indicated in Equation (26) to strike a balance between hardware efficiency and accuracy.

$$c_3 = c_4 = \begin{cases} \frac{1}{2^7}, & y < 1.125 \text{ or } y \geq 1.875 \\ \frac{1}{2^5}, & 1.125 \leq y < 1.875 \end{cases}. \quad (26)$$

Given c_3 is equivalent to c_4 , error compensation is needed only at L3, as L4's calculations also benefit from L3's error compensation in the multi-layer computation. For hardware implementation simplicity, we integrate the error compensation value c_3 with the original K_n^t to formulate KC_n^t values, as shown in Table 3.

5 Circuit Implementation

5.1 Multi-level Approximate Divider Architecture

The values for all $KC_n^t = K_n^t + c_n^t$ for $n = 2, 3, 4$ can be readily derived from the first 4 digits of y 's mantissa, as illustrated in Table 3. This design takes advantage of the fact that these values are exclusively powers of 2, simplifying the calculation of f_{ec}^n through straightforward shift and addition operations. The architecture employs the concept of **Partial Sums (PS)** for level-wise configuration. Depending on the value of KC_n^t , Level 2 involves one PS configuration, with two configurations required for both Level 3 and Level 4. Conversely, Level 1 demands three PS

Table 3. Relationship between the First 4 Digits of y Mantissa and KC_n^t

$y[1:4]^*$	KC_2^t	KC_3^t	KC_4^t
0000	$KC_2^0 = 2^{-3}$	$KC_3^0 = 2^{-4} + 2^{-7}$	$KC_4^0 = 2^{-5}$
0001			$KC_4^1 = -2^{-5} + 2^{-8}$
0010		$KC_3^1 = -2^{-5} + 2^{-6}$	$KC_4^2 = 2^{-5} - 2^{-8}$
0011			$KC_4^3 = -2^{-5} + 2^{-7}$
0100	$KC_2^1 = -2^{-4}$	$KC_3^2 = 2^{-4} + 2^{-6}$	$KC_4^4 = 2^{-5} - 2^{-7}$
0101			$KC_4^5 = -2^{-6} - 2^{-8}$
0110		$KC_3^3 = 0$	$KC_4^6 = 2^{-6} + 2^{-8}$
0111			$KC_4^7 = -2^{-6}$
1000	$KC_2^2 = 2^{-4}$	$KC_3^4 = 2^{-4}$	$KC_4^8 = 2^{-6}$
1001			$KC_4^9 = -2^{-7} - 2^{-8}$
1010		$KC_3^5 = 2^{-6}$	$KC_4^{10} = 2^{-7} + 2^{-8}$
1011			$KC_4^{11} = -2^{-7}$
1100	$KC_2^3 = 0$	$KC_3^6 = 2^{-5} + 2^{-6}$	$KC_4^{12} = 2^{-7}$
1101			$KC_4^{13} = -2^{-8}$
1110		$KC_3^7 = 2^{-7}$	$KC_4^{14} = 2^{-8}$
1111			$KC_4^{15} = 0$

*The first 4 digits of y mantissa.

configurations to compute Equation (15). The complex circuit design for computing the mantissa is outlined in Figure 5, where $p_{n,i}$ represents the i th partial sum production at level n .

The architecture utilizes only multiplexers and shifters to generate all PS outputs. For example, Figure 6 illustrates the hardware setup for Level 3, where the two PSs, $p_{3,1}$ and $p_{3,2}$, correspond to the two components of the $KC_3^t(x - y)$ computation result. Here, all shifters execute arithmetic right shifts. In this configurable design, each level is linked to a control bit, which, when set to 1, triggers the calculations for that particular level. When the control bit is set to 0, the output of that level will be fixed to 0, which prevents bit flipping and dynamic power consumption of the subsequent 3-2 compressors.

5.2 Design for Runtime Configurability

We introduce a new tree structure to efficiently calculate PS. This structure uses 3-to-2 compressor units (represented by brackets in Figure 7) to implement equation Equation (14). This design allows the circuit to dynamically switch between different approximate levels. Each dot in the figure represents a partial sum, while squares and triangles represent the sum and carry outputs of the compressors, respectively. Dotted arrows indicate direct connections that pass PS forward.

Notably, the tree features four output nodes, encased in red rectangles, aligning with approximation levels 1, 2, and levels 3 and 4, which include the error compensation term c_3 . This design allows the system to efficiently operate at lower approximate levels without necessarily traversing the entire tree. This optimization reduces delay in the critical path and enhances overall efficiency. The inherent flexibility of this approach makes it well-suited for applications demanding runtime configurability.

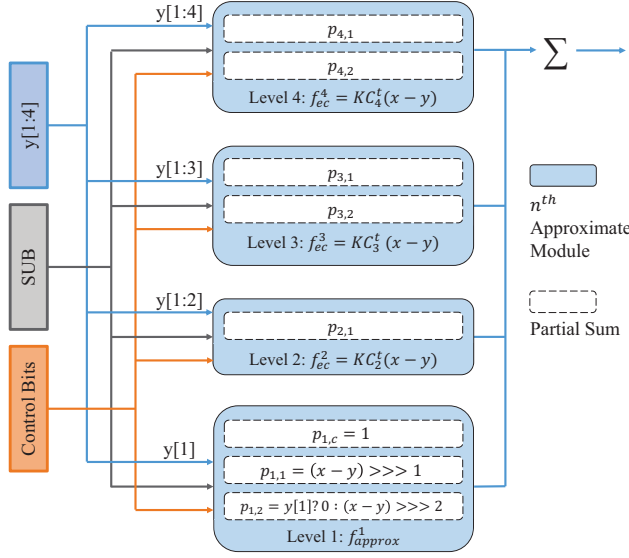


Fig. 5. Detailed architecture of the multi-level mantissa computation.

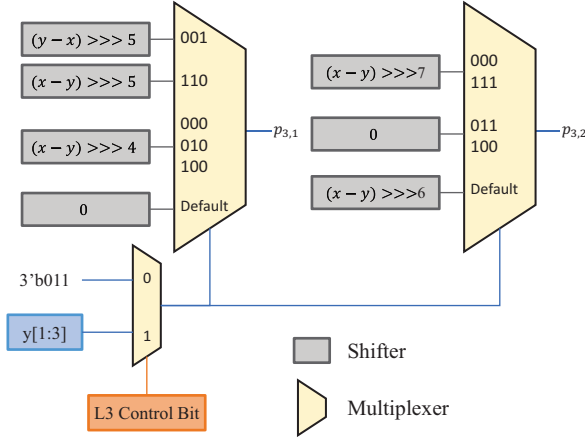


Fig. 6. Detailed implementation of L3 mantissa computation.

5.3 Truncation Mechanism

To further reduce hardware overhead, we can explore truncating the mantissa bits. In this subsection, we will evaluate truncation's computational errors and leverage the analysis to guide the choice of truncated bits.

Let $z = x - y$ and its truncation to the b th bit as z_b . For example, if $x - y = z = 0.101011\dots$, then $z_3 = 0.101$. Additionally, we denote the difference between z and z_b as $\hat{z}_b = z - z_b < 2^{-b}$. Thus, the approximate division result with truncation is

$$x/y \approx k_n^t \times (x - y) + 1 \approx k_n^t z_b + 1. \quad (27)$$

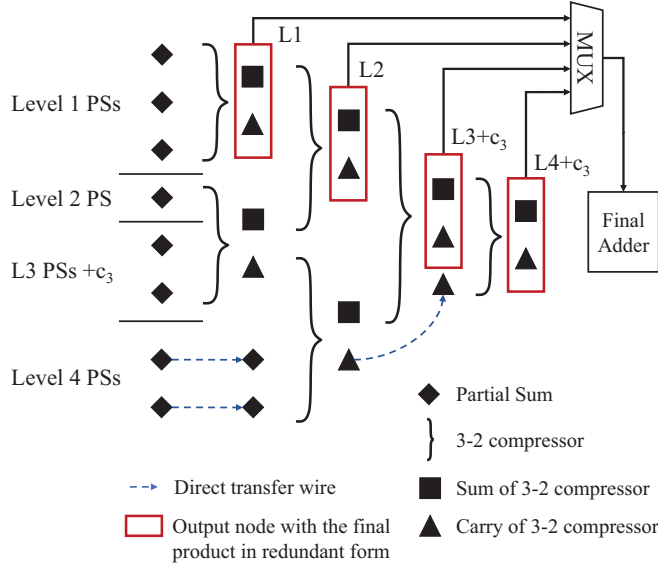


Fig. 7. Partial sum reduction tree structure for the proposed divider with runtime configurability.

Similar to the prior error analysis, we use MSE to assess truncation's computational error:

$$MSE_b = \int_1^2 \int_1^2 (x/y - k_n^t z_b - 1)^2 dx dy = \int_1^2 \int_1^2 \left[\left(\frac{1}{y} - k_n^t \right) (x - y) + k_n^t \hat{z}_b \right]^2 dx dy. \quad (28)$$

Then, the incremental MSE due to truncation compared to the original MSE is

$$\begin{aligned} \Delta MSE &= MSE_b - MSE = \int_1^2 \int_1^2 [(k_n^t \hat{z}_b)^2 + 2k_n^t \hat{z}_b \left(\frac{1}{y} - k_n^t \right) (x - y)] dx dy \\ &= (\hat{z}_b)^2 \int_1^2 (k_n^t)^2 dy + \hat{z}_b \int_1^2 k_n^t \left(\frac{1}{y} - k_n^t \right) (3 - 2y) dy \end{aligned} \quad (29)$$

Ignoring the second-order small term in ΔMSE and considering the use of conservative estimates in subsequent estimations, we can achieve:

$$\Delta MSE \approx \hat{z}_b \int_1^2 k_n^t \left(\frac{1}{y} - k_n^t \right) (3 - 2y) dy. \quad (30)$$

Based on the value of c_n^t in error analysis, with $1/y - k_n^t < 2^{-4}$ and $k_n^t < 1/y$, the estimated ΔMSE is

$$\Delta MSE < \hat{z}_b \int_1^2 \frac{1}{y} \left(\frac{1}{2^4} \right) (3 - 2y) dy \leq \frac{1}{2^{b+4}} (3 \ln 2 - 2) < \frac{1}{2^{b+7}}. \quad (31)$$

According to Equation (31), we can determine the number of truncation bits based on its impact on the error. For example, if we want to limit truncation's $RMSE$ increase to no more than 5% of the original $RMSE$, with $MSE > 2^{-15}$ in this design, we can set $\frac{1}{2^{b+7}} < 2^{-15} \times 0.05 \times 0.05$ to require $b = 16$. It's important to note that this is a conservative estimate, and practical truncation may need fewer than 16 bits to achieve a 5% $RMSE$ increase. Section 6.1 presents experimental data that further validates the relationship between the number of truncated bits and PACE calculation accuracy. This additional evidence supports the accuracy of our truncation analysis.

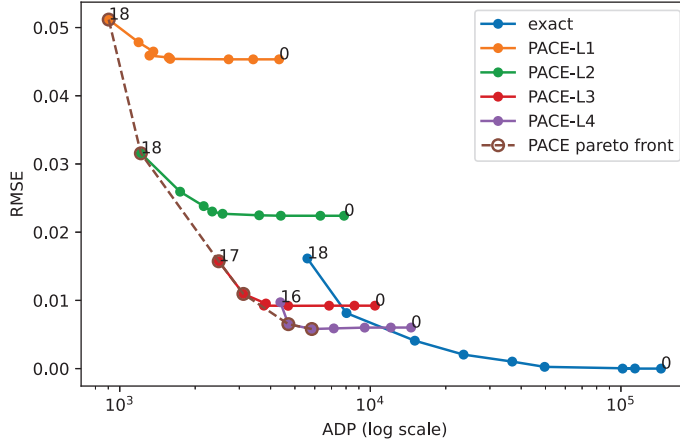


Fig. 8. Comparison on ADP v.s. RMSE for truncated PACE and exact divider.

6 Experimental Results

In a comprehensive evaluation, we compared the proposed PACE divider across four accuracy levels (L1–L4)* with truncated exact dividers and several state-of-the-art approximate dividers from prior works. The evaluation includes tests on accuracy, resource usage, and performance.

When measuring accuracy, 10,000 data points were uniformly sampled within the normalized range $[1, 2)$ for both divisors x and dividends y , given that PACE and other dividers haven't implemented any unconventional approaches in the exponent part. Four error metrics were used for evaluation: **Mean Error (ME)**, **Mean Absolute Error (MAE)**, **Mean Relative Error Distance (MRED)**, and RMSE.

For hardware-related metrics, all designs were implemented in Verilog HDL. Syntheses were performed using Synopsys Design Compiler with UMC's 40-nm library, using consistent design constraints and options across all implementations.

6.1 Comparison with Truncated Exact Divider

To assess the effectiveness of PACE's truncation mechanism, we compared it to a baseline implementation where we simply truncated the mantissa of the exact 32-bit floating-point divider from the Synopsys DesignWare library.

We sequentially applied truncation, removing 0, 4, 8, 12, 14, 15, 16, 17, and 18 last significant bits from the mantissa part. This sequence prioritizes reducing computational complexity during the initial, flatter portion of the error curve, while still maintaining accuracy as the curve becomes steeper. The maximum number of truncation bits for each PACE level is ultimately limited by the minimum bit requirement of the error compensation mechanism.

The tradeoff between accuracy (RMSE) and resource utilization (ADP) for PACE and the exact divider with varying truncation widths is visualized in Figure 8. First, we investigated the relationship between the number of truncated bits and PACE calculation accuracy. Our analysis revealed a consistent error trend across all approximation levels: truncating fewer than 12 bits (retaining at least 12 bits in the mantissa) resulted in minimal RMSE increase, while further truncation led

*The term PACE L{n} refers to its highest accuracy level, as any PACE divider ($L \geq 2$) can always be reconfigured to run at a lower level to reduce delay.

to an exponential rise in RMSE. This confirms the accuracy of Equation (31) and demonstrates the value of truncation analysis in determining the optimal truncation bits.

Subsequently, we compared the resource utilization of truncated PACE and truncated exact dividers. A dotted curve roughly represents the Pareto front, highlighting the most efficient points across different PACE levels and truncation configurations. For instance, at an RMSE of approximately 0.016, PACE(L3) with 17-bit truncation achieves a significant 44.3% reduction in ADP compared to the exact divider using 18-bit truncation. PACE L1 and L2 with 18-bit truncation provide flexibility for design by enabling a tradeoff between slightly reduced accuracy and significant ADP savings. Notably, some higher-level truncated versions can even outperform the lower-level 32-bit versions. This creates a broader Pareto front for PACE, offering more options for divider selection.

The results of other error metrics lead to similar conclusions. The figure highlights PACE's strength in balancing resource usage with accuracy compared to a simply truncated exact divider. Moreover, truncation doesn't compromise PACE's configurability. We can utilize a truncated PACE(L4) and dynamically switch to a lower level during runtime.

Given that not all dividers from prior work possess a truncation feature, in the rest of this article, we compare PACE with other dividers without truncation, using the 32-bit version for all.

6.2 Comparison with Prior Work

For comparison, we used the exact 32-bit FP divider and multiplier IPs from the Synopsys DesignWare library. We also evaluated several state-of-the-art approximate dividers,[†] including TruncApp [29], FaNZeD [28], LEAD [27], and QIAD [17].

The experimental setup remained consistent with the details provided in Section 6.1 for a fair comparison. PACE's minimal error bias (measured by ME) makes it suitable for use in cascaded operations where errors can accumulate. And results presented in columns 3–5 of Table 4, where the minimal error metric results are highlighted in bold, clearly demonstrate that the accuracy of PACE improves with increasing approximation levels. Notably, PACE outperforms FaNZeD and TruncApp even at the L1 approximation level for MAE and MRED metrics. Similarly, PACE surpasses LEAD from L2 onwards. While QIAD achieves high accuracy, it comes at the cost of significantly larger area/power consumption and higher latency. Conversely, PACE achieves comparable accuracy at L4 with substantially lower area/power and latency.

Table 4 (columns 6–9, with optimal two values in bold) summarizes the hardware resource and performance results evaluated by Synopsys Design Compiler, including area, delay, power consumption, and normalized area-delay product (Norm. ADP). The exact divider from the Synopsys DesignWare library serves as the baseline, with its results normalized to 1 for comparison. Even at its most resource-intensive level (L4), PACE exhibits significant advantages. Compared to the exact divider, PACE achieves a 70% reduction in area and a 66% improvement in speed. Furthermore, PACE(L4) surpasses even the exact multiplier IP in terms of size and speed.

PACE also surpasses other approximate dividers in terms of area, delay, and power consumption. Notably, PACE(L4) achieves similar accuracy as QIAD, but with only a third of the area and three times the speed. While FaNZeD boasts a smaller area than PACE(L4), its error rates are significantly higher, around 5–6 times greater. Among prior works, LEAD is the most balanced in terms of area, delay, power, and accuracy. However, compared to PACE(L4), LEAD is 70% larger, 120% slower, and incurs nearly 4× larger errors.

PACE demonstrates a remarkable improvement in ADP metrics, achieving over a 90% reduction compared to the exact divider. This signifies a superior tradeoff between accuracy and re-

[†]Some dividers were proposed as fixed point dividers but can be easily converted to FP dividers by removing the LOD modules and the barrel shifters.

Table 4. Accuracy Evaluation and Hardware Resource Consumption for an Exact Multiplier, PACE, and Several Prior Dividers (32-bit FP Version)

Designs	ME↓	MAE↓	MRED↓	RMSE↓	Area↓ (μm^2)/ ratio	Delay↓ (ns)/ ratio	Power↓ (mW)/ ratio	Norm. ADP
DW* Divider	-	-	-	-	15143/1.00×	9.54/1.00×	20.16/1.00×	1.000×
DW Multiplier	-	-	-	-	7574/0.50×	4.63/0.48×	6.62/0.32×	0.242×
PACE L1	-0.008	0.028	0.026	0.045	1981/0.13×	2.18/0.23×	1.21/0.06×	0.030×
PACE L2	-0.001	0.015	0.015	0.022	3152/0.21×	2.49/0.26×	2.02/0.10×	0.054×
PACE L3	0.001	0.006	0.006	0.009	3514/0.23×	2.97/0.31×	2.67/0.13×	0.072×
PACE L4	0.002	0.004	0.004	0.006	4509/0.30×	3.22/0.34×	3.56/0.18×	0.101×
FaNZeD [28]	0.005	0.030	0.029	0.039	1660/0.11×	1.47/0.15×	0.93/0.05×	0.017×
TruncApp [29]	0.085	0.085	0.083	0.095	5644/0.37×	5.48/0.57×	4.61/0.23×	0.214×
LEAD [27]	0.000	0.021	0.021	0.028	7815/0.52×	7.31/0.77×	3.82/0.19×	0.395×
QIAD [17]	-0.002	0.006	0.006	0.007	13594/0.90×	9.89/1.04×	11.82/0.59×	0.930×

*Synopsys DesignWare Library

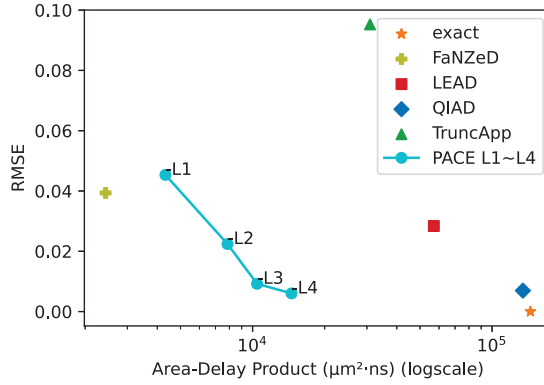


Fig. 9. Comparison on ADP v.s. RMSE among PACE and prior works.

source utilization, as illustrated in Figure 9. Notably, compared to LEAD, PACE(L4) boasts a 75% improvement in ADP and a 76.57% increase in accuracy based on RMSE. Furthermore, PACE delivers accuracy comparable to QIAD, but with a significantly lower ADP, requiring only 10.75% of the resources.

7 Application Level Comparison

Previous studies have illustrated the efficiency of using approximate dividers in realistic applications [3, 17, 20, 25, 27, 28]. To compare the performance of our proposed divider with others, we integrated them into three multimedia application algorithms that heavily depend on division operations: K-means color quantization, JPEG, and Gaussian Filter.

Figure 10 gives an overall structure of the evaluation workflow. To capture realistic input data characteristics, we intercept division operations during the execution of the application programs. An amount of 100,000 number pairs are randomly sampled from each application, representing their specific input patterns and realistic workloads. Based on the exported data and the netlist files previously synthesized by the Design Compiler, we conduct gate-level timing simulation using VCS. The generated switching activity results, along with the netlists, are finally fed into PrimeTime PX for precise power analysis and profiling.

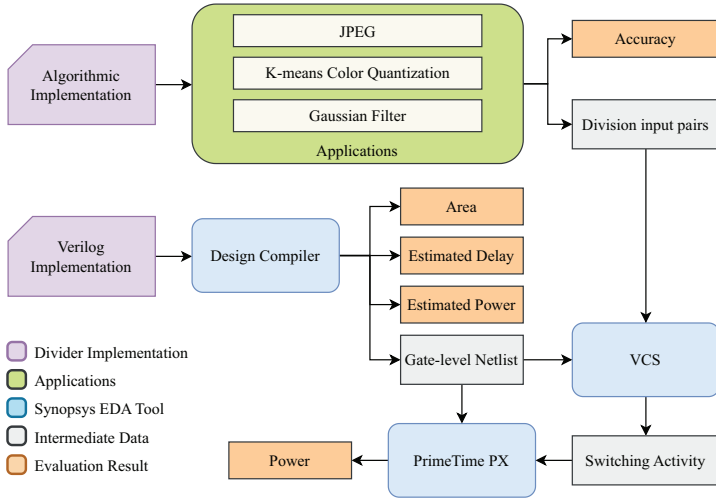


Fig. 10. Evaluation workflow for application level comparisons.

7.1 Accuracy

In K-means Color Quantization, we incorporated the approximate dividers into the standard cluster averaging operation performed in each K-means iteration, as done by Ratnaparkhi et al. [27]. This operation relies heavily on division. For JPEG image compression, similar to Saadat et al. [28], we replaced the element-wise division within the **Discrete Cosine Transform (DCT)** step with the approximate dividers. This substitution affects the quantization of DCT coefficients, a crucial step in JPEG compression. And in Gaussian filter, divisions implemented with approximate dividers are mainly used to compute the two-dimensional Gaussian function applied to each pixel for image smoothing. For these algorithms that rely on approximate division as a crucial intermediate result, the quality of the application output is directly linked to the accuracy and bias of the approximate division. Metrics such as MAE, MRED, and RMSE measure relative error, while ME evaluates unbiasedness. Generally, smaller error values and lower bias lead to better application output quality. These applications are evaluated on the Caltech101 dataset [16], which contains 102 categories and 9,144 images in total.

Figure 11 provides a glance at the visual quality of a sample image processed by the selected three multimedia applications, using either the exact divider or our proposed PACE divider.

To quantify the impact on image quality for comparison, we measured the difference in **Peak Signal-to-Noise Ratio (PSNR)**, between images processed with approximate and exact dividers. This difference is denoted as PSNR degradation, calculated as: $PSNR_{degradation} = PSNR_{accurate} - PSNR_{approx}$.

The statistics of PSNR degradation due to the use of approximate dividers on the Caltech101 dataset are demonstrated in Figure 12. It is obvious that a higher level of PACE leads to better image quality. In comparison with others, PACE(L2) outperforms most other approximate dividers for all three applications, except QIAD. At level 4, PACE exhibits minimal PSNR reduction, averaging only 0.09 dB, 0.63 dB, and 0.27 dB for K-means color quantization, JPEG, and Gaussian Filter, respectively. When compared to the most accurate QIAD divider among prior works, PACE(L4) achieves superior quality in all three applications, while requiring significantly fewer resources and incurring lower latency.



Fig. 11. Comparison between exact divider and proposed PACE(L4) applied in JPEG, K-Means Color Quantization, and Gaussian Filter.

Table 5. Power Consumption for Exact Divider, PACE, and Several Prior Dividers in Different Application Scenarios

Designs	Power↓ (mW)		
	JPEG	K-Means Color Quantization	Gaussian Filter
DW Divider	13.3	19.3	12.4
PACE L1	0.469	0.479	0.289
PACE L2	0.897	0.932	0.553
PACE L3	1.24	1.3	0.843
PACE L4	1.63	1.84	1.22
FaNZeD [28]	0.315	0.409	0.22
TruncApp [29]	3.44	3.62	2.08
LEAD [27]	5.79	4.83	3.3
QIAD [17]	9.75	23.5	5.46

7.2 Energy Efficiency

Table 5 presents the power measurement results, with the optimal two results highlighted in bold. Applications exhibit varying power consumption, but trends within each are consistent. The proposed PACE divider demonstrates impressive power efficiency. Compared to the exact

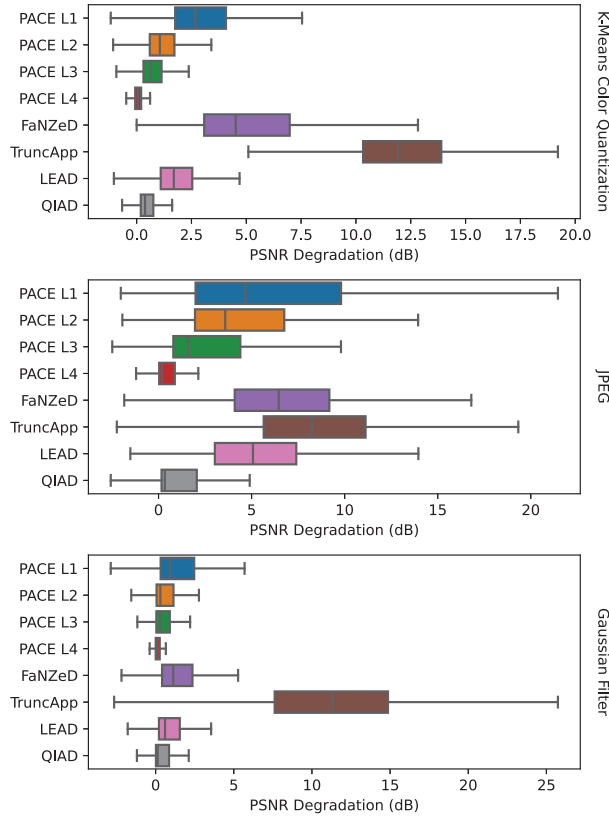


Fig. 12. Statistics of PSNR degradation due to approximate divisions on Caltech101 dataset.

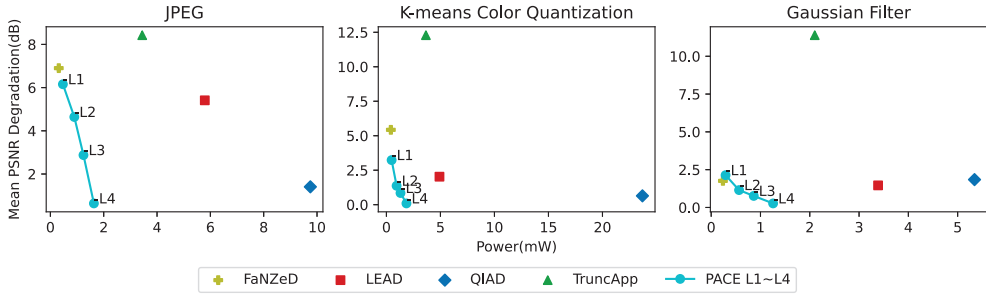


Fig. 13. Comparison on PSNR degradation v.s. Power consumption for dividers in realistic applications.

divider, PACE(L4) consumes only 12.3%, 9.5%, and 9.8% of its power for these tasks, respectively. PACE(L1) has comparable power consumption to FaNZeD, and even the most resource-intensive PACE(L4) outperforms other existing approximate dividers. This highlights PACE's efficiency in low-power design scenarios. Notably, the exact divider and QIAD consume significantly higher power, particularly in K-means color quantization. In contrast, PACE maintains consistent power usage across all applications.

Figure 13 offers a valuable comparison by visualizing both PSNR degradation and power consumption. This provides a more practical perspective on the performance of approximate dividers

instead of the rough estimations from Design Compiler. The PACE series consistently achieves a remarkable balance between accuracy (measured by minimal PSNR degradation) and energy efficiency (represented by low power consumption) across all three applications.

Among existing approximate dividers, FaNZeD exhibits a performance level similar to PACE(L1). While QIAD provides lower PSNR degradation than PACE L1 to L3, it leads to several times higher power consumption. Notably, PACE(L4) surpasses QIAD in both aspects. LEAD demonstrates a more balanced performance compared to TruncApp and QIAD, but it still falls behind the PACE series in achieving a superior tradeoff between accuracy and power efficiency.

The consistent performance of PACE across real-world multimedia applications demonstrates its stability and effectiveness. These results highlight PACE's wide applicability and potential for various division scenarios, offering a compelling alternative to traditional exact dividers that require high resource usage.

8 Discussion and Limitations

Although PACE has significant advantages over state-of-the-art approximate dividers in terms of approximate accuracy, configurability, and low hardware overhead, it still has some shortcomings:

- **Accuracy Improvement.** While PACE employs k_n^t , a power-of-2-based approximation of $1/y$, to simplify hardware implementation, it's important to note that k_n^t actually approximates $(0.5y^2 - 2y + 2.5)$. This limitation prevents continuous accuracy improvement through higher approximation levels, ultimately restricting PACE's accuracy to a level below that of a precise divider.
- **Unbiasedness Design.** To address potential bias in the approximate algorithm, we considered the impact of k_n^t 's approximation of $1/y$. Since k_n^t is inherently biased ($k_n^t < y_n^t$), multiplying by $(x - y)$ helps to symmetrize the deviation around 1. However, due to the actual range of x/y being $(1/2, 2)$, complete unbiasedness remains unattainable, as evidenced by the experimental data in Section 6.2.
- **Inefficiency for Low Levels.** While PACE(L4) offers exceptional accuracy with minimal hardware overhead, PACE(L1) may not be the most efficient choice for applications with a high tolerance for division errors. Compared to FaNZeD [28], PACE(L1) exhibits slightly higher overhead in terms of area, latency, and energy consumption. This suggests that other approximate dividers may be more suitable for scenarios where accuracy is less critical.

9 Conclusion

This work introduces PACE, a novel piece-wise approximate floating-point divider. Combining approximation and error correction, the proposed approach creates a multiplier-free, multi-level circuit with runtime configurability and adjustable accuracy. Compared to existing dividers, PACE offers a superior balance between accuracy and resource usage. Application-level evaluations show over 87.7% energy savings with PACE compared to an exact divider, with a marginal decline in output quality.

References

- [1] 2019. IEEE Standard for Floating-Point Arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)* (2019), 1–84. DOI: <https://doi.org/10.1109/IEEESTD.2019.8766229>
- [2] Neelam Arya, Teena Soni, Manisha Pattanaik, and G. K. Sharma. 2021. READ: A fixed restoring array based accuracy-configurable approximate divider for energy efficiency. *Integration* 76 (2021), 1–12. DOI: <https://doi.org/10.1016/j.vlsi.2020.08.002>
- [3] K. J. N. S. Bhargav, Sairam Paliseti, and Madhav Rao. 2021. A newton raphson method based approximate divider design for color quantization application. In *Proceedings of the 2021 18th International SoC Design Conference (ISOCC'21)*. IEEE, 115–116. DOI: <https://doi.org/10.1109/ISOCC53507.2021.9613961>

- [4] G. Bradski. 2000. The OpenCV library. *Dr. Dobbs's Journal of Software Tools* (2000).
- [5] Chuangtao Chen, Weikang Qian, Mohsen Imani, Xunzhao Yin, and Cheng Zhuo. 2021. PAM: A piecewise-linearly-approximated floating-point multiplier with unbiasedness and configurability. *IEEE Transactions on Computers* 71, 10 (2021), 2473–2486.
- [6] Linbin Chen, Jie Han, Weiqiang Liu, and Fabrizio Lombardi. 2015. Design of approximate unsigned integer non-restoring divider for inexact computing. In *Proceedings of the 25th Edition on Great Lakes Symposium on VLSI*. 51–56. DOI : <https://doi.org/10.1145/2742060.2742063>
- [7] Linbin Chen, Fabrizio Lombardi, Paolo Montuschi, Jie Han, and Weiqiang Liu. 2017. Design of approximate high-radix dividers by inexact binary signed-digit addition. In *Proceedings of the on Great Lakes Symposium on VLSI 2017*. 293–298.
- [8] Jorge Cortes, Sonia Martinez, Timur Karatas, and Francesco Bullo. 2004. Coverage control for mobile sensing networks. *IEEE Transactions on Robotics and Automation* 20, 2 (2004), 243–255.
- [9] Chuliang Guo, Li Zhang, Xian Zhou, Weikang Qian, and Cheng Zhuo. 2020. A reconfigurable approximate multiplier for quantized CNN applications. In *Proceedings of the 2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC'20)*. IEEE, 235–240.
- [10] D. L. Harris, S. F. Oberman, and M. A. Horowitz. 1997. SRT division architectures and implementations. In *Proceedings of the 13th IEEE Symposium on Computer Arithmetic*. 18–25. DOI : <https://doi.org/10.1109/ARITH.1997.614875>
- [11] Soheil Hashemi, R. Iris Bahar, and Sherief Reda. 2016. A low-power dynamic divider for approximate applications. In *Proceedings of the 53rd Annual Design Automation Conference*. 1–6. DOI : <https://doi.org/10.1145/2897937.2897965>
- [12] T. H. Haveliwala. 2003. Topic-sensitive PageRank: A context-sensitive ranking algorithm for Web search. *IEEE Transactions on Knowledge and Data Engineering* 15, 4 (2003), 784–796. DOI : <https://doi.org/10.1109/TKDE.2003.1208999>
- [13] Mohsen Imani, Ricardo Garcia, Andrew Huang, and Tajana Rosing. 2019. CADE: Configurable approximate divider for energy efficiency. In *Proceedings of the 2019 Design, Automation and Test in Europe Conference and Exhibition (DATE'19)*. IEEE, 586–589. DOI : <https://doi.org/10.23919/DAT.2019.8715112>
- [14] Honglan Jiang, Leibo liu, Fabrizio Lombardi, and Jie Han. 2019. Low-power unsigned divider and square root circuit designs using adaptive approximation. *IEEE Transactions on Computers* 68, 11 (2019), 1635–1646. DOI : <https://doi.org/10.1109/TC.2019.2916817>
- [15] Honglan Jiang, Francisco Javier Hernandez Santiago, Hai Mo, Leibo Liu, and Jie Han. 2020. Approximate arithmetic circuits: A survey, characterization, and recent applications. *Proceedings of the IEEE* 108, 12 (2020), 2108–2135.
- [16] Li Fei-Fei, Rob Fergus, and Pietro Perona. 2007. Learning generative visual models from few training examples: An incremental Bayesian approach tested on 101 object categories. *Computer Vision and Image Understanding* 106, 1 (2007), 59–70. DOI : <https://doi.org/10.1109/CVPR.2004.383>
- [17] Hao Liu, Ming-Jiang Wang, and Ming Liu. 2023. QIAD: A quadratic interpolation approximate divider. *IEICE Electronics Express* 20, 11 (2023), 20230167–20230167. DOI : <https://doi.org/10.1587/elex.20.20230167>
- [18] Weiqiang Liu, Fabrizio Lombardi, and Michael Shulte. 2020. A retrospective and prospective view of approximate computing [point of view]. *Proceedings of the IEEE* 108, 3 (2020), 394–399.
- [19] Joshua Yung Lih Low and Ching Chuen Jong. 2013. Non-iterative high speed division computation based on Mitchell logarithmic method. In *Proceedings of the 2013 IEEE International Symposium on Circuits and Systems (ISCAS'13)*. IEEE, 2219–2222. DOI : <https://doi.org/10.1109/ISCAS.2013.6572317>
- [20] Jackson Melchert, Setareh Behroozi, Jingjie Li, and Younghyun Kim. 2019. SAADI-EC: A quality-configurable approximate divider for energy efficiency. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (2019), 2680–2692. DOI : <https://doi.org/10.1109/TVLSI.2019.2926083>
- [21] John N. Mitchell. 1962. Computer multiplication and division using binary logarithms. *IRE Transactions on Electronic Computers* EC-11, 4 (1962), 512–517. DOI : <https://doi.org/10.1109/TEC.1962.5219391>
- [22] Sparsh Mittal. 2016. A survey of techniques for approximate computing. *ACM Computing Surveys* 48, 4, Article 62 (2016), 33 pages. DOI : <https://doi.org/10.1145/2893356>
- [23] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. 2018. *Handbook of Floating-Point Arithmetic* (2nd. ed.) . Birkhauser.
- [24] S. F. Obermann and M. J. Flynn. 1997. Division algorithms and implementations. *IEEE Transactions on Computers* 46, 8 (1997), 833–854. DOI : <https://doi.org/10.1109/12.609274>
- [25] James Oelund and Sunwoong Kim. 2023. ILAFD: Accuracy-configurable floating-point divider using an approximate reciprocal and an iterative logarithmic multiplier. In *Proceedings of the Great Lakes Symposium on VLSI 2023*. 639–644. DOI : <https://doi.org/10.1145/3583781.3590262>
- [26] Parhami and Behrooz. 2010. *Computer Arithmetic: Algorithms and Hardware Designs* (2nd. ed.) . Oxford University Press.
- [27] Omkar G. Ratnaparkhi and Madhav Rao. 2022. LEAD: Logarithmic exponent approximate divider for image quantization application. In *Proceedings of the Great Lakes Symposium on VLSI 2022*. 437–442. DOI : <https://doi.org/10.1145/3526241.3530323>

- [28] Hassaan Saadat, Haris Javaid, and Sri Parameswaran. 2019. Approximate integer and floating-point dividers with near-zero error bias. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6. DOI : <https://doi.org/10.1145/3316781.3317773>
- [29] Shaghyegh Vahdat, Mehdi Kamal, Ali Afzali-Kusha, Massoud Pedram, and Zainalabedin Navabi. 2017. TruncApp: A truncation-based approximate divider for energy efficient DSP applications. In *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE), 2017*. IEEE, 1635–1638. DOI : <https://doi.org/10.23919/DATE.2017.7927254>
- [30] Chenyi Wen, Haonan Du, Zhengrui Chen, Li Zhang, Qi Sun, and Cheng Zhuo. 2024. PACE: A piece-wise approximate and configurable floating-point divider for energy-efficient computing. In *Proceedings of the 2024 Design, Automation and Test in Europe Conference and Exhibition (DATE'24)*.
- [31] Chenyi Wen, Ying Wu, Xunzhao Yin, and Cheng Zhuo. 2023. Approximate floating-point FFT design with wide precision-range and high energy efficiency. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. 134–139.
- [32] Ying Wu, Chuangtao Chen, Weihua Xiao, Xuan Wang, Chenyi Wen, Jie Han, Xunzhao Yin, Weikang Qian, and Cheng Zhuo. 2024. A survey on approximate multiplier designs for energy efficiency: From algorithms to circuits. *ACM Transactions on Design Automation of Electronic Systems* 29, 1 (2024), 1–37.
- [33] Weihua Xiao, Cheng Zhuo, and Weikang Qian. 2022. OPACT: Optimization of approximate compressor tree for approximate multiplier. In *Proceedings of the 2022 Design, Automation and Test in Europe Conference and Exhibition (DATE'22)*. 178–183. DOI : <https://doi.org/10.23919/DATE54114.2022.9774628>
- [34] Qiang Xu, Todd Mytkowicz, and Nam Sung Kim. 2016. Approximate computing: A survey. *IEEE Design and Test* 33, 1 (2016), 8–22. DOI : <https://doi.org/10.1109/MDAT.2015.2505723>
- [35] Reza Zendegani, Mehdi Kamal, Arash Fayyazi, Ali Afzali-Kusha, Saeed Safari, and Massoud Pedram. 2016. SEERAD: A high speed yet energy-efficient rounding-based approximate divider. In *Proceedings of the 2016 Design, Automation and Test in Europe Conference and Exhibition (DATE'16)*. IEEE, 1481–1484. DOI : https://doi.org/10.3850/9783981537079_0521

Received 26 June 2024; revised 15 September 2024; accepted 22 November 2024